

Using the ENFOR-SA tool

1 Basic requirements

This tool works on Linux-based systems only. The fundamental system requirements are:

1. Linux-based systems: Tested on Ubuntu 22.04.5 LTS, but it should work with newer or similar versions;
2. Python3: Tested in Python 3.10.14, but should work with closer versions too.

2 Setting up the ENFOR-SA tool

The ENFOR-SA repository is self contained. All requirements can be installed by following the next instructions.

1. Clone the repository:
 - a. `git clone https://github.com/rafaabt/ENFOR-SA.git`
 - b. `cd ENFOR-SA/pytorch`
2. Install the requirements in a virtual environment:
 - a. `python3 -m venv enforsa_ve`
 - b. `source enforsa_ve/bin/activate`
 - c. `python3 -m pip install -r requirements.txt`

3 Run a basic tests

All experiments must be run in the `ENFOR-SA/pytorch` directory:

- a. `cd ENFOR-SA/pytorch`
- b. Then test the tool: `python tests/test_gemmini.py`

(Note: it takes a while for the tests to start running when executed for the first time)

Once finished, the output should be as shown in the figure below.

```

- Gemmini config : CustomGemminiOSMediumSoCConfig - Gemmini-OS-DIM8
- Gemmini mode   : OS
- Gemmini DIM    : 8
- Verilated module: VMesh
- Fault level    : RTL

test-fi_eval Injections: 400
Output tensors with errors: 400
.test_matmul_os iterations: 1000 - failed: 0
.test_matmul_rtl iterations: 100 - failed: 0
..test_preload_then_flush iterations: 1000 - failed: 0
.
-----
Ran 5 tests in 0.153s

```

This output indicates that the systolic array performs matrix multiplication properly (this includes input preloading, input streaming through the PEs, and flushing the PE outputs), and that faults are being effectively injected.

4 Running fault injection experiments

Fault injection can be performed at different fault abstraction levels: **Software**, **RTL**, or **gate-level** injection. Additionally, we provide two ways of performing fault injections: **sequential** (**run_seq.sh** script) and **parallel** injection (**run_par.sh** script). Fault injection experiments are configured in [experiment scripts](#) (see examples in the folder [python/experiments](#)).

To run the fault injection scripts, follow the next steps (A, B, C and D):

A. [PROVIDED] Configure your experiment script. The experiment scripts define basic parameters for the fault injection experiments. Among other parameters, the scripts define:

1. `$experiment`: An experiment name (will be the report folder name);
2. `$gemmini_config_key`: A systolic array configuration (examples provided);
3. `$fault_model`: The fault injection mode (options are “sw”, “rtl”, and “gl”);
4. `$experiment_description`: a brief description of the experiment;
5. `$sim_alias`: used as the part of the report files names;
6. `$fault_list`: A fault list file (stored in the folder [python/fault_lists](#));
7. `$injections`: The number of faults to inject;
8. `$batch_size`: The input batch size (ideal depends on the PC);
9. `$nbatches`: The number of batches to inject;
10. `$seed`: Seed used to select random inputs for injection.

B. [PROVIDED] Open the main fault injection script (**run_seq.sh** or **run_par.sh**): select the models and the layers to inject, as shown below:

```
1 # Enables the models to inject
2 models=("ResNet18"
3         "ResNet50"
4         #"ResNeXt101_32X8D"
5         #"ResNeXt101_64X4D"
6         ...)
7
8 # Defines the layers to inject
9 ResNet18_target_layers=(0 1 2 10 11 12 17 18 19)
10 ResNet50_target_layers=(0 1 2 5 10 15 20 25 30 35 40 45 50 51 52)
11 ResNeXt101_32X8D_target_layers=(0 1 2 20 30 40 50 60 70 80 90 101 102 103)
12 ResNeXt101_64X4D_target_layers=(0 1 2 20 30 40 50 60 70 80 90 101 102 103)
13 ...
14
15 # Enables the required injection layers
16 target_layers=(ResNet18_target_layers
17                ResNet50_target_layers
18                #ResNeXt101_32X8D_target_layers
19                #ResNeXt101_64X4D_target_layers
20                ...)
21
22 # Runs the sequential fault injection mode
23 for ((i=0; i<${#models[@]}; i++)); do
24     ./scripts/run/run_sequential.sh "${models[i]}" "${target_layers[@]}"
25 done
```

D. [PROVIDED] For RTL injection, customized desired fault targets can be selected. Faults can be configured to hit varying fault targets (e.g., input registers, output accumulator and control signals), layers, and PE and bit positions. To select specific targets, do as follows:

- I. Open the file `src/flist/fi_target.py`;
- II. Customize the desired filters, according to the many examples provided;
- III. In the function `setup_target()`, set `fault_target=<desired targets>`.

C. Run the injection scripts:

- I. To run sequential injection, hit: **./run_seq.sh**
- II. To run parallel injection, hit: **./run_par.sh**

If everything works well, you should see a progress bar as the fault injection loop is processed, as shown below.

```
(base) rafaelbt@PTB-5CG3105P21:~/ENFOR-SA/pytorch$ ./run_seq.sh

[Running with args]:
-model ResNet18 -config OSDIM8 -output reports/ResNet18/debug/sequential -bsize 8 -batches 20 -layer 0 -injections 100
as sim-xyz -gold False -seed 42

[Setting threads]:
- Inter: 2
- Intra: 2
- Device: cpu

[InstrumentedModel]:
- DNN model: ResNet18
- FI layer: 0
- FI mode: Sequential
- FI level: RTL

[Loading Gemmini extension]:
- Config: CustomGemminiOSMediumSoCConfig - Gemmini-OS-DIM8
- Mode: OS
- conf.DIM: 8
- Layer: conv1
- Replaced: QuantizedConvReLU2d(3, 64, kernel_size=(7, 7), stride=(2, 2), scale=0.028605546802282333, zero_point=0, pad

Processing fault list: 25% 25/100 [00:01<00:04, 17.98iter/s]
```

Once the simulation finishes, the report files are stored in the folder `reports/<dnn model name>/<$experiment>`. The report file names are determined by the concatenation of the `$alias`, `$gemmini_config_key`, and `$seed` parameters set in the experiment script. **[Important]** The report files are cumulative. If you run the injection again with the same experiment name, alias, seed and array config, the results accumulate in the same file. If necessary, you must delete the experiment folder before starting a new run.

5 Analyze the generated data

The simulation results can be seen in the *reports* folder, for each model and experiment runs. The *trace* folder contains the results for each injected fault, for each input, layer, and fault position in the systolic array. The column *sdc1* is asserted if the fault is critical (i.e., the Top-1 label != Top-1 golden label).

We provide simple pandas scripts to process the fault trace and do data analysis. Measuring the global AVF:

```
./eval.sh reports/ResNet18/exp-rtl-v1/sequential/trace/sim-xyz-s0-OSDIM8.csv
```

Estimating the parallel approach effectiveness: to do so, we compare the fault injection results from the sequential approach with the parallel approach. A jupyter notebook is provided to facilitate analysis.

```
jupyter notebook notebooks/analysis.ipynb
```